

Desarrollo robusto y validación estricta de tipos con Pydantic

Base teórica — Bloque 1 · Masterclass Equipo 4

1. El problema: Python confía demasiado

Un type hint como `edad: int` es una nota para humanos y editores, pero Python no la comprueba en tiempo de ejecución. Si una función espera un entero y recibe un texto, Python no se queja al momento — el error aparece más adelante, en un punto del programa que no tiene relación directa con la causa real.

```
def registrar_pasajero(edad: int): return edad * 2
registrar_pasajero("no soy un número") # no explota aquí... explota más adelante
```

2. La solución: Pydantic como inspector automático

Pydantic es una librería que valida y fuerza tipos de datos usando los type hints ya existentes en el código. Cuando se crea una instancia de un modelo, Pydantic revisa cada campo: si el tipo no coincide pero es convertible (por ejemplo, el texto "30" en un campo `int`), lo convierte automáticamente; si no es convertible, lanza un error claro indicando exactamente qué campo falló y por qué.

3. Conceptos clave

Concepto	Qué hace
BaseModel	Clase base de la que se hereda para definir el "esquema" de datos
Type hints	Definen el tipo esperado de cada campo (<code>str</code> , <code>int</code> , <code>list[str]</code> ...)
Coerción de tipos	Convierte automáticamente tipos compatibles (<code>"20"</code> → <code>20</code>)
ValidationError	Excepción lanzada cuando un dato no encaja ni es convertible
Field()	Añade restricciones: mínimos, máximos, longitud, valores por defecto
@field_validator	Lógica de validación personalizada para un campo concreto
Modelos anidados	Un <code>BaseModel</code> puede contener otro <code>BaseModel</code> como campo

4. Ejemplo mínimo

```
class PasajeroGalactico(BaseModel):
    nombre: str = Field(..., min_length=1, max_length=50)
    edad: int = Field(..., gt=0, le=1000)
    especie: Literal["humano", "vogon", "androide"]
    @field_validator("nombre")
    @classmethod
    def
```

```
nombre_no_vacio(cls, v): if v.strip() == "": raise ValueError("el nombre no puede estar vacío") return v.strip().title()
```

5. ¿Dónde se usa en la vida real?

APIs: FastAPI usa Pydantic internamente para validar cada request y response. **Configuración:** validar variables de entorno o archivos YAML/JSON antes de arrancar una aplicación. **Pipelines de Machine Learning:** comprobar que los datos de entrada tienen la forma correcta antes de pasarlos a un modelo.

6. Ficha de criterio ético

La validación estricta de tipos no es neutral: las reglas que definimos (longitudes mínimas, patrones de texto, formatos "válidos") reflejan supuestos sobre cómo deberían verse los datos, y pueden excluir casos legítimos que no encajan en ese molde — por ejemplo, un validador de nombres pensado solo para el alfabeto latino puede rechazar nombres con otros alfabetos, o un patrón de teléfono diseñado para un país puede bloquear números internacionales válidos. Un pipeline que descarta silenciosamente los registros que no pasan la validación puede introducir sesgos sistemáticos en los datos que llegan a un modelo de ML.

7. Coste computacional

Validar datos tiene un pequeño overhead frente a no validar nada, pero en la inmensa mayoría de casos (APIs, configuración, pipelines de datos) es insignificante frente al beneficio de detectar errores pronto. La excepción son bucles de altísima frecuencia, donde conviene validar una sola vez al entrar los datos y no en cada iteración interna.